

piral into an unmaintainable code if we iterate on it without a strategically important vision.

Leave the current permissions system as-is and build a parallel system

Instead, build a parallel Declarative Policy-based system alongside it to use for custom roles.

Pros:

- Faster to design and build a new system than to do a large-scale refactor of the existing system.
- Auth team can own this new system entirely.

Cons:

- Maintaining 2 systems
- Each new "regular" permission added needs a parallel addition to the custom roles system. This makes it difficult to have feature parity between custom roles and default roles.
- Replacing our existing RBAC system with custom roles (an eventual goal of the custom roles feature) is more difficult with this approach because it requires retiring the legacy permissions system.

Bundle existing permissions into custom permissions

Use "custom permissions" for the custom roles API.

Pros:

- Faster to design and build a new system than to do a large-scale refactor of the existing system.
- Auth team can own these new bundled permissions

Cons:

- Bundling permissions is less granular; the goal of custom permissions is to enable granular access.
- Each new "regular" permission added needs a parallel addition to the bundled permissions for custom roles. This makes it difficult to have feature parity between custom roles and default roles.

Glossary

- RBAC: Role-based access control; "a method of restricting network access based on the roles of individual users." RBAC is the method of access control that GitLab uses.
- Default roles: the 5 categories that GitLab users can be grouped into: Guest, Reporter, Developer, Maintainer, Owner (documentation). A default role can be thought of as a group of permissions.
- Declarative Policy: code library used by GitLab to define our authorization logic.

- Permissions: a specific ability that a user with a Role has. For example, a Developer can create merge requests but a Guest cannot. Each row listed in the permissions documentation represents a "permission" but these may not have a 1:1 mapping with a Declarative Policy ability.
An ability is how permissions are represented in the GitLab codebase.
- Access level: integer value representing a default role, used for determining access and calculating inherited user access in group hierarchies (documentation).

Resources

- Custom Roles MVC announcement
- Custom Roles lunch and learn notes
- Discovery on auto-generating documentation for permissions.

SECTION: engineering/architecture/design-documents/pipelines_table_redesign/_index.md

{{< engineering/design-document-header >}}

Summary

Associated Epic: 17330: Pipelines Table Redesign and GraphQL Migration

The Pipelines Table is a critical component of GitLab's CI/CD experience, appearing in three key locations throughout the application:

1. The main Pipelines page
2. The Commit Pipelines tab
3. The Merge Request Pipelines tab

The current implementation uses REST API and displays all information at once, including pipeline mini graphs for each pipeline. This has led to significant performance issues and a suboptimal user experience.

As the migration from REST to GraphQL for the Pipelines Table is already planned, this design document proposes combining this migration with a comprehensive UI redesign. The redesign will separate essential pipeline information from detailed data (like mini graphs and job information) using a progressive disclosure pattern, while the GraphQL implementation will enable more efficient data fetching and establish the foundation for future real-time updates.

By tackling both efforts simultaneously, we can significantly reduce development costs by avoiding temporary solutions and rework. This approach will improve initial load performance by reducing payload size and rendering complexity, enhance the user experience through better information hierarchy, and establish the technical architecture needed for eventual real-time pipeline status updates all while maintaining feature parity for critical pipeline information.

Business Impact

User Impact

The proposed redesign and migration will significantly improve the GitLab CI/CD experience by:

- Improving Performance: Reducing initial load time by 50%+, translating directly to increased developer productivity
- Enhancing Information Access: Implementing progressive disclosure to show essential information first while making details available on demand
- Streamlining Troubleshooting: Prioritizing failed jobs and actionable items in the expanded view to reduce MTTR
- Addressing Current Issues: Resolving multiple open issues related to Pipelines Table functionality, addressing long-standing user pain points
- Preparing for Real-time Updates: Establishing the foundation for future instantaneous pipeline status updates

Business Value

This initiative delivers substantial business value through:

- Infrastructure Cost Savings: 30-40% reduction in CI/CD-related server load by eliminating polling and reducing payload sizes
- Developer Efficiency: Significant time savings across millions of daily pipeline interactions through faster load times and improved information hierarchy
- Competitive Positioning: Reinforcing GitLab's market position by addressing key performance concerns
- Technical Debt Elimination and Prevention: Eliminating significant existing technical debt while avoiding future rework by proactively implementing modern patterns and technologies

Risk Mitigation

The implementation strategy includes several elements to mitigate potential business risks:

- Feature Flag Approach: The phased rollout behind a feature flag allows for controlled testing and monitoring, reducing the risk of disruption to critical workflows.
- User Preference Toggle: Providing users with the ability to switch between implementations ensures that those who rely heavily on the current interface can continue their work without interruption while adapting to the new design.
- Feature Parity: Maintaining feature parity for all critical pipeline information ensures that users don't lose functionality during the transition.
- Comprehensive Metrics: Detailed performance and user satisfaction metrics will be collected to validate improvements and identify any areas requiring adjustment.

ROI Calculation

Based on preliminary estimates:

| Investment

| Return

|

	Engineering effort: Partial allocation over ~4-5 milestones (approx 30% of engineers' time)
	30-40% reduction in CI/CD-related server resources
	Product design effort: Partial allocation over ~2 milestones
	Measurable improvement in pipeline interaction efficiency
	QA and validation effort: Partial allocation over ~1-2 milestones
	Reduction in CI/CD-related support tickets
	Improved user satisfaction scores
	Foundation for future real-time capabilities

Motivation

GraphQL Migration

The primary motivation for migrating the Pipelines Table from REST to GraphQL is to enable real-time updates for pipeline status information:

- Enable Real-time Updates: GraphQL subscriptions will allow us to deliver instantaneous pipeline status changes to users without inefficient polling mechanisms.
- Reduce Server Load: The current polling approach requires frequent, full-payload requests that unnecessarily consume server resources, particularly in environments with active CI/CD usage.
- Eliminate N+1 Query Issues: The current REST implementation suffers from N+1 query patterns when retrieving pipeline data, creating database performance bottlenecks. GraphQL's ability to batch and optimize queries will significantly reduce these database inefficiencies.
- Improve Filtering Performance: The GraphQL implementation will enable more efficient pipeline filtering by moving filter operations closer to the data source, reducing unnecessary data transfer and processing.
- Optimize Network Traffic: GraphQL allows precise specification of required data, significantly reducing payload sizes compared to REST endpoints that return fixed data structures.
- Technical Alignment: Most new GitLab features are being built with GraphQL, making this migration strategically aligned with the platform's technical direction and architecture.
- Future Scalability: As GitLab continues expanding its CI/CD capabilities, GraphQL provides a more flexible and scalable query language for accessing increasingly complex pipeline data structures.

UI Redesign

The primary motivation for redesigning the Pipelines Table UI is to address critical performance issues caused by including the pipeline mini graph in the parent API request.

- Excessive Initial Payload: The current implementation includes detailed stage and job data for the mini graph in the initial request, significantly increasing payload size and processing time for each pipeline.
- Information Architecture Improvement: The current approach presents too much information at

once (Hick's Law), making it difficult for users to quickly identify pipeline status. A redesign allows us to prioritize the most important information.

- Unnecessary Data Loading: Most users only need to view detailed mini graph information for a small subset of pipelines, yet we currently load this data for all visible pipelines.
- Progressive Disclosure Opportunity: By moving the mini graph and other detailed information to an expandable view loaded with a secondary request, we can drastically improve initial load performance while still providing access to all pipeline details.
- Enhanced Pipeline Details: Moving detailed information to a secondary request enables us to build a more robust Pipeline Details component that can include richer visualizations, comprehensive failed jobs information, and more detailed metrics without impacting initial page performance.

Previous Work and Issues Addressed

This initiative builds upon previous efforts while comprehensively addressing identified UX pain points. By combining the GraphQL migration with UI redesign, we can make progress on:

Previous Migration Attempts

Several attempts to address pipeline table performance through GraphQL migration have not gained sufficient traction as standalone initiatives. While these efforts targeted individual components, our proposal addresses all three instances of the pipeline table throughout the application:

- Epic 11080: Migrate MR pipelines tab to GraphQL
- Epic 16831: Migrate pipelines table to GraphQL
- Issue 223264: Move pipelines page to GraphQL
- Issue 223267: Move pipelines table filter search to GraphQL

These previous efforts highlighted the need for pipeline table improvements but faced challenges in prioritization when treated as separate initiatives.

UX Issues by Complexity

The redesign also provides an opportunity to address several long-standing UX issues in a coordinated effort:

High (Weight 3)	Medium (Weight 2)	Low (Weight 1)
321517 Improve table layout	432373 Add created-at time	435814 Add missing tooltips
327900 Improve mini-graph UX	329513 Reevaluate tab headings	354074 Add skeleton loader
300256 Improve "Triggered by me"	365616 Align buttons with design system	
339651 Add configuration options		

Our user research indicates that pipeline performance and usability issues are consistently reported as significant pain points by users, yet addressing them has been repeatedly deprioritized in favor of new feature development.

Goals

The primary goals of this initiative are:

Performance Improvement

- Reduce initial page load time for the Pipelines Table by at least 50%
- Decrease the size of the initial data payload by at least 60%
- Improve client-side rendering performance by reducing initial DOM element count
- Eliminate polling in favor of efficiently designed GraphQL queries and future subscription capability

User Experience Enhancement

- Improve information hierarchy by separating essential pipeline information from detailed data
- Create a more intuitive interface that allows users to quickly scan pipeline statuses
- Maintain feature parity for all critical pipeline information
- Achieve at least 80% user satisfaction in usability testing

Technical Architecture

- Complete migration from REST to GraphQL for all pipeline table functionality
- Design and implement a component architecture that supports future real-time updates
- Create a separation between list-level data and detailed pipeline information

Development Efficiency

- Implement both the UI redesign and GraphQL migration in a single coordinated effort
- Avoid creating temporary solutions or technical debt
- Develop behind a feature flag to enable controlled testing and rollout

Non-Goals

- Implementation of real-time updates (though laying groundwork is in scope)
- Redesign of the pipeline mini graph
- Changes to the underlying CI/CD execution engine
- Migration of other CI/CD features to GraphQL (job logs, runner management, etc.)
- Changes to pipeline creation or configuration workflows

Proposal

The proposed solution involves a complete redesign of the Pipelines Table coupled with a migration from REST to GraphQL. The new design will adopt a progressive disclosure pattern, showing only essential information at the list level while providing users with the ability to expand individual pipeline entries to access detailed information.

This approach will utilize two separate GraphQL queries:

1. A lightweight initial query that loads only essential pipeline data for the list view
2. A second, more detailed query that is only triggered when a user expands a specific pipeline

By combining these two initiatives into a single coordinated effort, we can achieve significant performance improvements while enhancing usability and setting the stage for future real-time updates, all while minimizing development costs.

Information Architecture

The new design will separate pipeline information into two levels:

1. List View (Essential Information)

- Pipeline status and finished timestamp
- Pipeline ID and name
- Branch, commit, and MR information
- Author information
- Quick actions

2. Expanded View (Detailed Information)

Appears when user expands a pipeline row

- Possibly pipeline mini graph visualization (see Pipeline Mini Graph Placement decision)
- Failed jobs information
- Job actions
- Duration metrics
- Available artifacts
- Detailed commit information
- Configuration details

Design and Implementation Details

Implementation Approach

The implementation will be phased as follows:

Phase 1: Design and Infrastructure (1-2 milestones)

- Conduct product design sessions to finalize UI/UX requirements [Design]
- Create designs for both list view and expanded details [Design]
- Define information hierarchy and user interactions [Design]
- Introduce feature flag infrastructure [BE/FE]
- Update user preference schema to add usePipelinesListView (see User Preference Implementation) [BE]

Phase 2: Foundation and Architecture (1 milestone)

- Update GraphQL schema based on finalized designs [BE]
- Stub out query resolvers [BE]
- Build and test the details query [BE]
- Base component structure for list view [FE]
- Base component structure for details view [FE]

Phase 3: Implementation and Integration (1-2 milestones)

- Implement query resolvers and logic [BE]
- Implement user preference toggle for all 3 locations [FE]
- Build UI child components [FE]
- Query integration [FE]
- Conduct performance testing [BE/FE]
- Conduct user testing [Design/FE]

Phase 4: Rollout and Monitoring (1 milestone)

- Gradually enable feature flag
- Monitor performance metrics and toggle usage patterns
- Gather user feedback through dedicated feedback channels
- Adjust default toggle settings based on feedback and metrics
- Complete rollout when adoption and satisfaction metrics meet targets

Contributors

The implementation of this initiative will require:

- 3-4 Frontend engineers
- 1-2 Backend engineers
- 1-2 UX/UI designers

Key Design Decisions

List View Format

<details>

<summary>Details</summary>

Decision DRI: Product Design

Decision: [DECISION PENDING] Selection of the optimal list view format for the redesigned Pipelines Table.

Context: The current Pipelines Table uses a traditional table format with fixed columns, which presents all information at once and becomes visually complex with the pipeline mini graphs. A redesigned list view needs to balance information density with readability and usability, focusing on the most essential information while providing access to details when needed.

Options to consider:

- Row-based list view
- Card-based view
- Other

</details>

Detail View Interaction

<details>
<summary>Details</summary>

Decision DRI: Product Design

Decision: [DECISION PENDING] Selection of the optimal interaction method for accessing detailed pipeline information.

Context: With the introduction of the progressive disclosure pattern in the redesigned Pipelines Table, we need an effective way to display detailed pipeline information on demand. The component should provide sufficient space for comprehensive details while maintaining context and not disrupting the overall workflow.

Options to consider:

- Expandable row revealing details panel beneath the row
- Side drawer showing related pipeline details
- Other

</details>

User Preference Implementation

<details>
<summary>Details</summary>

Decision DRI: Product Design, Engineering

Decision: [DECISION PENDING] Whether to implement a user preference toggle to allow users to switch between the current Pipelines Table and the new redesigned version.

Context: As we transition from the current REST-based implementation to the GraphQL redesign, we need to provide users with a smooth migration path that allows them to access both versions during the transition period.

Implementation:

The user preference will be controlled with a GraphQL mutation:

```
graphql
mutation updateUsePipelinesListView($usePipelinesListView: Boolean) {
  userPreferencesUpdate(
    input: { usePipelinesListView: $usePipelinesListView }
  ) {
    userPreferences {
      usePipelinesListView
    }
  }
}
```

}

The frontend will include a toggle component similar to the Work Items toggle that will:

- Display a badge indicating whether the new design is enabled
- Provide a popover with toggle control to switch between views
- Include a link to a dedicated feedback issue
- Handle preference persistence through the GraphQL mutation

Rollout Strategy:

1. Initial Phase: The feature will be enabled for internal GitLab users only, with the toggle defaulting to "Off"
2. Beta Phase: As confidence grows, the feature flag will be enabled for a wider audience, still with the toggle defaulting to "Off"
3. Gradual Rollout: Based on feedback and performance metrics, the toggle will gradually default to "On" for new users
4. Full Rollout: Eventually, the toggle will be removed and the redesigned implementation will become the standard

This approach enables users to try the new design while maintaining access to the familiar interface, facilitates feedback collection, provides a fallback mechanism, and creates a smoother transition experience.

</details>

Pipeline Mini Graph Placement

<details>

<summary>Details</summary>

Decision DRI: Product Design, Engineering

Decision: [DECISION PENDING] Whether to include the pipeline mini graph in the details view or focus on failed jobs and actionable items instead.

Context: The mini graph requires fetching extensive job data (including dozens of passed jobs with names and statuses) that is often unnecessary for the user's workflow.

Options:

1. Include Mini Graph in Details View

- Provides visual representation of pipeline stages
- Maintains familiar visualization element
- Requires fetching data for all jobs, including non-actionable ones

2. Focus on Failed Jobs and Actionable Items

- Prioritizes information users need to take action
- Reduces data requirements by focusing on relevant jobs
- Shows stage summary without individual passed job details
- Potentially more useful for troubleshooting workflows

Next Steps:

- Research user workflows to identify most frequently needed information
- Analyze query patterns to measure reduction in data requirements
- Test alternative presentations focusing on actionable information

Initial Recommendation: Focus primarily on failed jobs and actionable items in the details view, with a simplified stage summary that doesn't require fetching data for every passed job.

</details>

GraphQL Query Structure

<details>

<summary>Details</summary>

Decision DRI: Engineering

Decision: Implement two separate GraphQL queries and prepare for future subscription model

Context: GraphQL allows for precise data fetching and real-time updates through subscriptions.

Benefits:

- Optimized initial payload
- On-demand loading of detailed information
- Foundation for real-time updates
- Better separation of concerns in frontend code

Alternatives Considered:

- Single comprehensive query with all data - Rejected due to performance concerns
- Multiple fragmented queries - Rejected due to increased request overhead

</details>

Pipeline Details Schema

<details>

<summary>Details</summary>

Decision DRI: Engineering

Decision: [PENDING UI DESIGNS] Determine the optimal GraphQL schema for pipeline details based on finalized UI designs.

Context: The schema for the secondary query will be driven by UI requirements to ensure we only fetch data needed for the expanded details view.

Process:

- Complete UI designs for the expanded pipeline details view
- Identify all data elements required by the design
- Develop schema that efficiently retrieves only necessary information
- Validate schema against performance goals

Considerations:

- Balance between comprehensive information and query efficiency
- Prioritize data for troubleshooting and actionable items
- Structure for future extensibility and real-time updates
- Support for all three pipeline table contexts

Next Steps:

- Finalize expanded details UI mockups
- Map UI elements to required data fields
- Draft initial GraphQL schema
- Review with stakeholders

Implementation Target: A focused schema that retrieves exactly what the UI needs without unnecessary data fetching.

</details>

GraphQL Queries

The migration from REST to GraphQL for the Pipelines Table will involve creating two primary query patterns:

1. Lightweight list queries that retrieve only essential pipeline information for each context
2. A detailed query that fetches comprehensive information for a specific pipeline

<details>

<summary><h4>List Queries</h4></summary>

The list queries will be optimized for performance, retrieving only the data necessary for the initial list views. This approach significantly reduces the payload size compared to the current REST implementation. We will implement pagination and basic filtering capabilities from the start to ensure users do not lose current functionality, with additional filters to be added iteratively as needed.

We'll define a common fragment for all list queries to ensure consistency:

```
graphql
fragments/pipelinelistfields.fragment.graphql
```

```

fragment PipelineListFields on Pipeline {
  id
  iid
  detailedStatus {
    ...Cilcon
  }
  createdAt
  finishedAt
  user {
    id
    name
    avatarUrl
    webUrl
  }
  commit {
    id
    shortId
    webUrl
  }
  mergeRequest {
    id
    webUrl
    reference
  }
  retryable
  cancelable
}

```

Then we'll implement three specific queries for each context where the Pipelines Table appears:

Project Pipelines Query:

```

graphql
queries/projectpipelines.query.graphql
query getProjectPipelines(
  $projectPath: ID!
  $first: Int
  $after: String
  $filters: PipelineFilterInput
) {
  project(fullPath: $projectPath) {
    id
    pipelines(first: $first, after: $after, filters: $filters) {
      pageInfo {
        hasNextPage
        endCursor
      }
      nodes {
        ...PipelineListFields
      }
    }
  }
}

```

```

        Project-specific fields will be added here
    }
}
}
}

```

Merge Request Pipelines Query:

```

graphql
queries/mergerequestpipelines.query.graphql
query getMergeRequestPipelines(
  $projectPath: ID!
  $mergeRequestId: ID!
  $first: Int
  $after: String
) {
  project(fullPath: $projectPath) {
    id
    mergeRequest(iid: $mergeRequestId) {
      id
      pipelines(first: $first, after: $after) {
        pageInfo {
          hasNextPage
          endCursor
        }
        nodes {
          ...PipelineListFields
          MR-specific fields will be added here
        }
      }
    }
  }
}
}

```

Commit Pipelines Query:

```

graphql
queries/commitpipelines.query.graphql
query getCommitPipelines(
  $projectPath: ID!
  $sha: String!
  $first: Int
  $after: String
) {
  project(fullPath: $projectPath) {
    id
    commit(sha: $sha) {
      id

```

```

    pipelines(first: $first, after: $after) {
      pageInfo {
        hasNextPage
        endCursor
      }
      nodes {
        ...PipelineListFields
        Commit-specific fields will be added here
      }
    }
  }
}

```

</details>

Details Query

The details query will be established once the UI designs for the expanded view are complete, as noted in the Pipeline Details Schema decision.

UI Component Architecture

[This structure will evolve once designs are complete]

```

shell
ci/pipelineslist/
... components/
. . . PipelinesListView.vue  Container component
. . . PipelineListItem.vue  Individual pipeline row component
. . . PipelineDetails.vue  Expandable details component
. . . renderless/
. . . . ProjectPipelinesQuery.vue  Renderless component for project pipelines query
. . . . MergeRequestPipelinesQuery.vue  Renderless component for merge request pipelines query
. . . . CommitPipelinesQuery.vue  Renderless component for commit pipelines query
... graphql/
. . . fragments/
. . . . pipelinelistfields.fragment.graphql
. . . queries/
. . . . projectpipelines.query.graphql
. . . . mergerequestpipelines.query.graphql
. . . . commitpipelines.query.graphql
. . . . pipelinedetails.query.graphql
. . . subscriptions/  For future real-time updates
. . . . . pipelinestatuses.subscription.graphql
. . . . . pipelinedetails.subscription.graphql
... constants.js
... utils.js

```

Alternative Solutions

GraphQL Migration Without UI Redesign

Approach: Migrating to GraphQL while maintaining the current UI design.

Pros:

- Provides some performance benefits
- Allows incremental improvements
- Lower risk of user experience disruption

Cons:

- Requires building GraphQL queries that match current REST payloads
- Creates technical debt when UI is eventually redesigned
- Doesn't address information hierarchy or cognitive load issues
- Misses opportunity for combined implementation efficiency

Metrics

To establish baseline performance measurements and set clear targets for improvement, we've gathered the following metrics across the locations where the Pipelines Table appears:

Current Performance Baselines

Location	Metric
Current Value	Target
-----	-----
Main Pipelines Page	Average network response time (no filtering)
1.35 seconds TBD	
Main Pipelines Page	Average network response time (with trigger author/status filter)
2.38 seconds TBD	
Merge Request Pipelines Tab	Average network response time
1.64 ms TBD	

We'll measure the success of this initiative by comparing pre and post-implementation metrics across these dimensions, with specific improvement targets as outlined in the Goals section.

Query Performance Comparison

To thoroughly evaluate the performance improvements of our proposed changes, we've conducted a comparative analysis of three different query approaches using the same test dataset (project pipelines) in a controlled test environment:

Query Approach Comparison

Approach	Description
----------	-------------

Query Complexity		Server Response Time		Payload Size
Client Processing Time				

-----		-----		
-----		-----	-----	-----

Current REST Implementation Fetches all pipeline data including mini graphs in a single request				
High (N+1 queries for stages/jobs)		1.35 seconds		
256 KB	450 ms			
GraphQL Without UI Redesign GraphQL implementation that maintains the same data structure as current REST API				
Medium (Optimized queries but still fetching all data)		TBD		
TBD	TBD			
GraphQL With UI Redesign Two-tier GraphQL approach with separate list and details queries				
Low (Optimized for essential data only)		TBD		
TBD	TBD			

Detailed Query Analysis

Current REST Implementation

Endpoint: GET /api/v4/projects/:id/pipelines?perpage=20

- Complexity: Triggers multiple database queries per pipeline for stages and jobs
- Data Returned: Complete pipeline data including all stages and jobs
- Drawbacks: Significant overhead for information that may not be viewed

GraphQL Without UI Redesign

Endpoint: GraphQL API with comprehensive pipeline data query

- Complexity: Reduced database queries through batch loading
- Data Returned: Same comprehensive dataset as REST
- Expected Improvement: Some efficiency gains through GraphQL optimization

GraphQL With UI Redesign

Primary Endpoint: GraphQL API with lightweight pipeline list query

Secondary Endpoint: GraphQL API with detailed pipeline information query (on-demand only)

- Complexity: Minimal database queries for essential data only
- Data Returned: Only data required for initial list view, with details fetched separately
- Expected Improvement: Significant reduction in payload size and processing time

Testing Methodology

The performance metrics were gathered using the following methodology:

- Test environment: Production GitLab instance
- Test project: Mid-sized project with 20 pipelines (standard pagination size)
- Measurement tools: Browser Developer Tools, GitLab performance monitoring

- Metrics captured: Server processing time, payload size, client rendering time
- Sample size: Average of 20 requests per implementation

Expected Performance Improvements

Based on preliminary testing, we anticipate the following improvements with the GraphQL UI redesign approach:

Metric	Expected Improvement
Server Response Time	65-70% reduction
Initial Payload Size	75-80% reduction
Client Rendering Time	50-60% reduction
Time to Interactive	60-65% reduction

These expectations will be validated with comprehensive testing during implementation, with actual measurements to be added once available.

SECTION: engineering/architecture/design-documents/pipeline_execution_policy/_index.md

{{< engineering/design-document-header >}}

This document is a work in progress and represents the current state of the vision to allow users to enforce CI jobs to be run as part of project pipelines and enabling users to link/scope those jobs to projects using security policies and compliance frameworks.

Summary

Users need a single solution for enforcing jobs to be run as part of a project pipeline. They want a way to combine the flexibility of compliance framework pipelines with the simplicity of scan execution policies.

There are many cases that could be addressed using pipeline execution policies to define policy rules, but here are a few of the most common we've heard so far:

1. I want to include CI templates using include statements and be able to pull in templates such as .latest security scanner templates. See CI templates.
1. I want to enforce execution of a 3rd party scanner, so that I can capture additional security results and manage them alongside results from GitLab scanners in the vulnerability report.
1. I want to enforce a custom script/check for compliance.
1. I want to enforce scanners to analyze code stored in GitLab, but call out to a 3rd party CI tool to continue a pipeline. A common case here may be for users migrating to GitLab, but some teams still may be using 3rd party CI tools in the meantime.
1. I want to export results from scanners to a 3rd party tool.
1. I want to block MRs until a UAT in an external tool is completed (quality gate), so that I can ensure the quality of changes to my production code.
1. I want to standardize custom project badges that can quickly communicate to users in a

project which security policies are enabled and if pipelines are blocked by a given scan result, so that my security team and developer team can easily understand the security state of a project.

1. I want to create a custom badge in projects to communicate the security grade, so that team members are incentivized to keep their score high.

Motivation

Goals

1. Provide a proper replacement for compliance pipelines, with an objective of deprecating compliance pipelines in 16.9 and removal in 17.0.

1. Set up the architecture in a way that the feature can be enhanced to solve known compliance pipelines issues and customer pain points for enforcing scan execution globally across their organization.

Known compliance pipelines issues are:

- Unable to support a project with an external config file or no config file
- CI variables in the Compliance config can be overwritten
- Compliance jobs can be overwritten by target repository
- Conflict with top-level global keywords
- Must ensure compliance jobs are always run
- Prefilled variables are not shown when manually starting pipeline
- Need to unify Scan Execution Policies and Compliance Framework
- Compliance Framework not found or access denied
- Compliance pipeline jobs don't run in a specific situation due to GitLab only checking the project's pipeline configuration
- .pre/.post jobs should be optionally allowed in empty pipelines
- Allow compliance pipelines to hold constant any values that are set from an include file
- Run Compliance Framework pipeline in MRs that originate in forks of the CF-labeled project
- Compliance pipeline configuration is not exposed in mergedYAML

Proposal

Currently, security policies can include multiple scan actions. Each scan action will result in a CI job that will be injected in the project CI pipeline.

The new policy type Pipeline Execution Policy allows users to define custom CI jobs that will be injected into the project CI pipeline as well. We want to generalize the security policy approach to provide the same flexibility that compliance framework needs. The combination of the 2 features

means that security policies can be scope to compliance frameworks and enforce the presence of custom CI jobs.

Like Scan Execution Policies, Pipeline Execution Policy jobs can be scoped

to certain compliance frameworks applied to the project.

It should be possible to control when the policy jobs are enforced by using the existing workflow rules.

Users can leverage one of the predefined security-policy stages to position jobs in the

pipeline according to their needs.

Transitioning from compliance pipelines to the new feature should be as smooth as possible.

Design and Implementation Details

Pipeline Execution Policy MVC

The Pipeline Execution Policy MVC will allow the transition from compliance pipelines.

- It should be possible to add custom CI YAML to a security policy. For simplicity, the custom CI YAML should support the same configuration and follow the same schema as `.gitlab-ci.yml` files. The custom CI will be merged with the project CI when a pipeline starts.
- The security policy schema should allow the custom CI to be defined in an external file by allowing a project and file path to be added.
- Pipeline Execution Policies should execute custom CI YAML by creating jobs in isolated pipelines which are merged into the pipeline of the target projects.
- Organizations may require different CI configurations/templates to enforce globally for various use cases. Commonly, some projects focus on code and do not perform deployment. These projects may have different security/compliance requirements compared to other projects that facilitate the build and deployment steps. Pipeline execution policies must be able to support this need, allowing for variable configuration dependent on the type of project. Thus, it may be necessary to support more than one pipeline execution policy per level (e.g., more than one policy at the group level). However, we may be able to leverage policy scopes to limit the number of configurations enforced per level against a single project.
- When considering pipeline execution policy limits, another use case is hierarchical enforcement. Policies may be enforced globally (defined in one top-level group and enforcing all other groups/projects). Policies may also be enforced a tier lower in the business, such as via individual business units. Each BU may have unique requirements to satisfy. While we want to limit the opportunity for collision, we may be able to identify a solution that satisfies the need for top-down enforcement. Some customers have been exploring mechanisms for appending/extending policy enforcement down the tree/hierarchy. We could also start with lower limits, ensuring that top-level groups and sub-groups higher in the tree take precedence over projects or lower-tier subgroups.
- At minimum, Pipeline Execution Policy jobs should align with existing CI variable precedence. Ideally, Pipeline Execution Policy jobs should not get any user-defined variables except those defined in the group or project where the policy belongs.
- Jobs should be executed in a way that is visible to users within the pipeline and that will not allow project jobs to override the policy jobs. In Scan Execution Policies today, we utilize the index pattern `(-0,-1,-2,...)` to increment the name of the job if a job of the same name exists. This also gives some minor indication of which jobs are executed by a security policy. For Pipeline Execution Policy jobs, the same pattern should be utilized.
- Jobs coming from the policies should be marked as such in the database so that they can be distinguished, for example by using build metadata. This allows for different handling of jobs and the corresponding variables by the runner.
- Users should be able to define the stage in which their job will run, and Pipeline Execution Policies will have a method to handle precedence. For example, a security/compliance team may want to enforce jobs that run commonly after a build stage. The stages and jobs must not interfere with those defined by development teams once enforced by a Pipeline Execution Policy.
- Pipeline Execution Policies should allow for jobs to be enforced in projects that do not contain an existing CI configuration.

- To reduce complexity, the content of a Pipeline Execution Policy can only be an inclusion of a single CI file from a project.

MVC syntax example:

```
yaml
type: pipelineexecutionpolicy
name: PEP example
description: 'PEP example'
enabled: true
pipelineconfigstrategy: injectci
content:
  include:
    - project: namespace-path/project-path
      file: policyci.yml
      ref: main
```

Stages management strategy

We want users to be able to place jobs to run before or after certain CI stages of the project pipeline.

To achieve this, we want to introduce 2 reserved stages that can be used only by pipeline execution policies and injected into the project pipeline:

1. .pipeline-policy-pre stage will run at the very beginning of the pipeline, before the .pre stage.
1. .pipeline-policy-post stage will run at the very end of the pipeline, after the .post stage.

Injecting jobs in any of these stages is guaranteed to always work. Execution policy jobs can also be assigned to any standard (build, test, deploy) or user-declared stages. However, in this case, the jobs may be ignored depending on the project pipeline configuration.

We will try this approach as part of the experiment phase. We also discussed the following strategies that we might want to try:

1. Make security policy stages order take precedence

Pipeline execution policies can modify stages by redefining them using the stages keyword. The order of stages defined in the pipeline execution policy will take precedence over the order defined in the project CI.

If a pipeline execution policy wants to inject a job after the test stage, it can redefine stages as:

```
yaml
stages:
  - test
  - policyaftertest
```

This solution provides flexibility to users as they can define custom stages. The downside is that there will be no single source of truth for the order of stages anymore.

If a project CI defines a custom stage that is not defined in the pipeline execution policy, we don't know if it should run before or after the pipeline execution policy stages.

2. Introduce pre and post keywords

Security policies can use `pre[stagename]` and `post[stagename]` stages to inject jobs before or after certain stages. For example, `pretest` would run before the test stage.

This way the security policy are unlikely to disrupt the project CI.

The downside is that the policy author needs to be aware of the stages used by projects and policy stages can be skipped by renaming the test stage to `test2` for example.

3. Introduce advanced stage rules

An advanced API for pipeline execution policy stages allows to inject stages depending on the existence of other stages. Schema example:

```
yaml
compliancestages:
- stagename: qa
  after:
  - release
  if:
  - stageexist: deploy
```

This will provide flexibility and allows users to define the behavior for different project CI stage setups.

4. Run all jobs in a default stage

Jobs defined in a pipeline execution policy will run in the pipeline-policy stage. This stage will be injected after the test stage.

If the test stage doesn't exist, it will be injected after the build stage. If the build stage doesn't exist, it will be injected at the beginning of the pipeline.

Pipeline config strategies

There will be two strategies to handle the pipeline configuration with pipeline execution policies:

1. `injectci`: This strategy allows for the addition of custom CI configurations into the existing project pipeline without completely replacing the project's original CI configuration. It is suitable when the goal is to enhance or extend the current pipeline with additional steps, such as adding new security scans, compliance checks, or custom scripts.

1. `overrideprojectci`: This strategy completely replaces the project's existing CI configuration

with a new one defined by the pipeline execution policy. Ideal when the entire pipeline needs to be standardized or replaced, such as enforcing organization-wide CI/CD standards or compliance requirements. When multiple policies are applied to a project, if one policy uses the `overrideprojectci` strategy, this strategy will take precedence over all other policy strategies.

Context of the policy pipeline

For each policy pipeline attached to a project, we use the same context as the main pipeline when building jobs.

This includes the pipeline source, commit SHA, merge request / pipeline scheduler context, etc. We do this to ensure that the policy pipeline behaves the same way as the main pipeline. The policy authors

will know that a policy pipeline is not a separate pipeline, but a set of jobs that are merged into the project pipeline and use the same rules as in the main pipeline.

For example, the source of the main pipeline is passed to the policies when building them and their jobs

because policy pipelines need to know this information to be able to have these configs;

yaml

workflow:

rules:

- if: \$CIPIPELINESOURCE == 'schedule'
- if: \$CIPIPELINESOURCE == 'push'
- if: \$CIPIPELINESOURCE == 'merge-request-event'

Pipelines excluded from policy enforcement

Some pipeline sources are excluded from policy enforcement and pipeline execution policies are not injected into pipelines with these sources.

These pipeline sources are called "dangling";

`webide`, `parentpipeline`, `ondemanddastscan`, `ondemanddastvalidation`, `securityorchestrationpolicy`, `containerregistrypush`.

They are safe to skip from policy enforcement because they are either considered safe from the point of view of the content (on-demand DAST scans, SEP pipelines, etc.)

or because we enforce the policy upstream (e.g. in parent pipelines instead of child pipelines).

Links

- Pipeline Execution Policy Type
- Pipeline Execution Action (Custom CI YAML Support) for Scan Execution Policy Type

SECTION: engineering/architecture/design-documents/pipeline_mini_graph/_index.md

{{< engineering/design-document-header >}}

This blueprint serves as living documentation for the Pipeline Mini Graph. The Pipeline Mini Graph is used in various places throughout the platform to communicate to users the status of the relevant pipeline. Users are able to re-run jobs directly from the component or drilldown into said jobs and linked pipelines for further investigation.

Motivation

The Pipeline Mini Graph originally relied on REST for its functionality. We have updated the component to support GraphQL and are in the process of migrating all instances to this API. This documentation will act as the single source of truth (SSOT) for the refactor. Developers have found it challenging to contribute to the component due to the coexistence of two APIs, making it essential to simplify the code while ensuring compatibility with both REST and GraphQL.

Goals

- Improved maintainability
- Backwards compatibility
- Improved query performance
- Deprecation of REST support
- Real-time pipeline status updates

Non-Goals

- Redesign of the Pipeline Mini Graph UI

Proposal

To break down implementation, we are taking the following steps:

1. [COMPLETE] Separate the REST version and the GraphQL version of the component into 2 directories called `pipelineminigraph` and `legacypipelineminigraph`. This way, developers can contribute with more ease and we can easily remove the REST version once all apps are using GraphQL.
1. [COMPLETE] Optimize GraphQL query structure to be more performant.
1. [COMPLETE] Finish updating the newer component to fully support GraphQL
1. [COMPLETE] Roll out `cigraphqlpipelineminigraph` to globally enable GraphQL instances of the component.

Implementation Details

Issue Milestone Status
---- -
Move legacy files to new directory 17.1 ·
Move remaining legacy code 17.1 ·
Create README for PMG 17.1 ·

- | GraphQL Query Optimization | 17.1 | · |
- | Dedicated component for downstream pipelines | 17.1 | · |
- | Fetch Stage by ID | 17.2 | · |
- | Job Item | 17.2 | · |
- | Job Actions | 17.3 | · |
- | Rollout cigraphqlpipelineminigraph | 17.8 | · |
- | Migrate pipeline editor PMG to GraphQL instance | 17.8 | · |
- | Migrate commit page PMG to GraphQL instance | 17.8 | · |
- | Remove dead logic from PMG codebase | TBD | To Do |
- | Migrate MR PMG to GraphQL instance | TBD | To Do |

Design Details

Structure

All data for the pipeline mini graph is passed into the component. This data comes from various API calls throughout different apps which use the component. The data passed in is formatted to GraphQL structure within the entry file. All API calls which then occur in the component itself are GraphQL queries/mutations.

Properties

Name	Type	Required	Description
downstreamPipelines	Array	false	pipelines triggered by current pipeline
isMergeTrain	Boolean	false	whether the pipeline is part of a merge train
pipelinePath	String	false	pipeline URL
pipelineStages	Array	true	stages of current pipeline
upstreamPipeline	Object	false	upstream pipeline which triggered current pipeline

File Structure

plaintext

```

... pipelineminigraph/
.  ... downstreampipelines.vue
.  ... pipelineminigraph.vue << entry point
.  ... pipelinestagedropdown.vue
.  ... pipelinestages.vue
..... graphql/
.    ... fragments/
.      ... job.fragment.graphql
.    ... mutations/
.      ... jobcancel.mutation.graphql
.      ... jobplay.mutation.graphql
.      ... jobretry.mutation.graphql
.      ... jobunschedule.mutation.graphql
.    ...queries/
.      ... getpipelinestagejobs.query.graphql

```

Considerations

Properties

- isMergeTrain: This property is specific to the MR page and is used to display a message in the job dropdown to warn users that merge train jobs cannot be retried. This is an odd flow. Perhaps we should consider either having this data come from somewhere else within the pipeline mini graph, or living in the merge train widget itself. It is worth noting here that this boolean is not used for any other logic outside of displaying this message.

Future Improvements

- GraphQL Subscriptions
- Show downstream pipeline jobs
- Possible redesign

SECTION: engineering/architecture/design-documents/poly_repos/_index.md

{{< engineering/design-document-header >}}

This document is a work in progress and outlines a plan to support workflows that involve multiple repositories. We call this poly repos for short.

Summary

The "poly repos" architecture introduces "Change Sets" to GitLab, enabling organizations with multi-repository codebases (like Android Open Source Project) to manage cross-repository changes as a single unit.

This delivers critical workflow improvements by allowing teams to review, test, and merge related changes across multiple repositories simultaneously, with the ability to revert these changes together if needed, all while maintaining reproducible CI builds.

This capability directly addresses the complex integration challenges faced by enterprises working with distributed codebases, reducing development friction and improving software delivery reliability.

Motivation

Many customers have source code repositories that depend on other repositories. Android developers, for example, may work with a handful of repositories, but there are over a thousand repositories involved with building Android. A sample:

- platform/manifest - The central repository that defines all other repositories needed for a

build.

- platform/frameworks/base - Core Android framework
- platform/system/core - Low-level system components
- platform/bionic - Android's C library implementation

Currently GitLab only supports merge requests for a single project, but users should be able to work across projects. Users should be able to:

1. Push changes that span projects.
1. Review these changes together in one place.
1. Ensure CI passes with these changes.
1. Merge when all tests pass.
1. Revert the changes if something goes wrong.

Definition of terms

- Change Set - A group of proposed code changes that span multiple projects.
- Target Product - A group of projects that represents the target product for a Change Set.
For example, the Android source code is one example of a product consisting of hundreds of individual Git repositories.
- Manifest file - typically an XML configuration file that declares your app's package name, components, dependencies between them and their current revision.

Goals

1. Define how to identify projects that make up a target project.
1. Define how merges and reverts will work across multiple projects.
1. Define how to build changes across projects with a centralized pipeline.
1. Discuss how Code Review will work with poly repo changes.

Non-Goals

Proposal

Design and implementation details

What is a Change Set?

There are two main ways a Change Set could be represented:

- Patch Set: A group of individual Git patches that are applied to different projects.
- Grouped Merge Requests: A group of merge requests that cross project boundaries.

Grouped Merge Requests vs. Patch Set

In the GitLab model, merge requests could be grouped together into a logical set and represent the changes needed to be applied together. This has the advantage of working with existing workflows, such as code review, merge request pipelines, etc.

However, there are a number of disadvantages:

- Diffs have to be recomputed on the server every time the target branch changes. If there are many outstanding merge requests across many repositories, this could create a significant amount of load on the server. Note that this problem already exists for monorepos.
- Each change to a project requires creating a new branch for that project. Developers have to name and manage individual branches across many repositories.

Patch sets have a number of advantages:

- They are computed on the client side.
- They force developers to have good commit hygiene, keeping logical changes together.
- Merging and reverting individual patches might be more straightforward than reverting a large merge commit.

The main disadvantages:

- Using Patch Sets may reduce the quality of the code review experience. For example, the ability to view code coverage, run security scans, report quality metrics, view the entire content of a file, etc. may not work.
- It's not clear how CI jobs would be able to create reproducible builds. CI jobs would have to check out some revision and then apply the Patch Set. What revision should the job use?
- Updating a Patch Set may be cumbersome and result in poor user experience. If a reviewer requests changes, the user must recreate the patch set, which is kind of equivalent to squashing and overriding the branch history in a MR.
- There's not a great way to apply patch sets locally. According to this discussion there is no repo apply command that allows someone to apply all patches across projects.

How are Change Sets created?

There should be a number of ways a Change Set can be created:

1. From the UI
1. Using the API
1. Git push options

A change set could be created right before pushing the changes or even during the development time. It is a bucket where to add/remove MRs.

If a change set ID is allocated before pushing the changes we could use push options to pass the changeset ID:

```
bash
glab cset new --product=10 => CSET ID: 1234
```

```
cd ../proj-1
git push -o cset=1234 origin HEAD
```

```
cd ../proj-2
git push -o cset=1234 origin HEAD
...
```

This could be done automatically with glab:

```
bash
push changes to multiple repos using product ID=10 and allocate a CSET ID if doesn't exist
glab push --polyrepo --product=10 => CSET created: https://gitlab.com/-/csets/1234
```

```
or
glab push --polyrepo-manifest=../manifest.xml
```

Constraints of change sets

1. A merge request that belongs to a change set can only be merged together with other MRs in the change set.
1. A merge request can only be present in at most 1 change set at a given point in time.
1. A changeset can not be deleted because it represents point in time information of when certain MRs were merged. Much like merge requests cannot be deleted.

Known unknowns:

- Should a merge request belonging to a change set be allowed to be closed?
Should it instead be removed from the change set?
- Should modifying the change set (by adding or removing a merge requests) trigger a CI pipeline?
Technically it would cause the overall diff of the change set to change, invalidating the build.

Identify projects that make up a "product"

Epic

An organization (loosely defined as instance or group) can have multiple `products`. There could be multiple poly-repos in a given organization.

The "product" is orthogonal to the group/sub-group/project hierarchy. It can be a map of any arbitrary projects in the organization, not necessarily belonging to the same group.

- organization has many `products`
- a product has many projects/repos
- a product has many changesets
- a changeset has many MRs (implicitly project + ref)

Organizations aren't generally available yet, but a top-level group might

be considered a temporary container until organizations are available.

Usually a Change Set will involve a subset of the projects involved in a target project. Customers currently use a number of approaches to identify a product. For example:

1. Monorepos

- 1. A central repository with Git submodules
- 1. An Android manifest file

For the purpose of this discussion focusing on poly repos, we will ignore monorepos.

Git submodules

Git submodules allow you to include other Git repositories within your repository as subdirectories, while keeping commits separate. Key characteristics:

- 1. Embeds specific commits of external repositories into your main repository.
- 1. Changes to submodules are tracked by the parent repository as single-commit pointers.
- 1. Requires explicit commands to update (git submodule update).
- 1. Native Git feature, no extra tools required.
- 1. Each submodule maintains its own .git directory.

Advantages:

- 1. Direct Git feature requiring no additional tooling.
- 1. Simple conceptual model - repositories inside repositories.
- 1. Works well for stable dependencies that change infrequently.
- 1. Good for libraries and components with fixed versions.

Challenges:

- 1. Notorious for usability issues and learning curve.
- 1. Easy to forget to update or initialize submodules.
- 1. Difficult to work with when frequently changing multiple repositories.
- 1. Commands can be verbose and error-prone.

Android manifest file

The Android manifest approach uses an XML file (typically default.xml) to describe the repository structure, managed by Google's "repo" tool. Key characteristics:

- 1. Uses a manifest XML file that specifies repositories, branches, and paths.
- 1. Repositories are cloned side by side rather than nested.
- 1. Centrally managed through a single manifest repository.
- 1. Designed specifically for handling large, complex projects with many repositories
- 1. Uses the "repo" tool as a wrapper around Git.

Advantages:

1. Better suited for large-scale projects with many components.
1. More flexible for managing branch tracking across repositories.
1. Simplifies syncing multiple repositories to consistent states.
1. Supports more complex workflows like topic branches across repositories.
1. Provides high-level commands that handle multiple repositories at once.

Challenges:

1. Requires learning the "repo" tool and its commands.
1. Additional dependency on the repo tool.
1. Less universally supported in Git hosting platforms and tools.

Shortcomings

Here are some limitations of the Android manifest approach:

1. Flat Structure: The manifest primarily defines a flat collection of Git repositories to be checked out at specific revisions or branches. It doesn't inherently track which repository depends on which.
1. No Dependency Resolution: There's no automatic resolution of dependencies or sub-dependencies. The manifest simply lists all repositories that should be present.
1. Manual Coordination: Project maintainers manually determine which versions of repositories work together and update the manifest accordingly.
1. No Transitive Dependencies: If Repository A depends on Repository B, which depends on Repository C, the manifest doesn't automatically resolve this chain--all three would need to be explicitly included.
1. Groups: The manifest does have a concept of "groups" which allows categorizing repositories, but this is for organizational purposes and doesn't represent dependency relationships.

One customer generates lockfile to handle this with the command `repo manifest -r -o lockfile.xml`. A pseudo-structure of a lockfile could be:

yaml

repositories:

framework/base:

remote: aosp

commithash: a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0

dependson:

- system/core

- external/protobuf

description: "Android application framework"

system/core:

remote: aosp

commithash: b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0u1

dependson:

- external/libcutils

description: "Core Android system components"

external/protobuf:
remote: aosp
commithash: c3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0u1v2
dependson: []
description: "Protocol Buffers library"

external/libcutils:
remote: aosp
commithash: d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0u1v2w3
dependson: []
description: "Android C utility library"

device/google/pixel:
remote: google
commithash: e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0u1v2w3x4
dependson:
- hardware/qcom/display
description: "Pixel device-specific code"

hardware/qcom/display:
remote: qcom
commithash: f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0u1v2w3x4y5
dependson: []
description: "Qualcomm display HAL"

Iteration plan

As a thought experiment, let's suppose that we start on fully supporting poly-repos workflows. Here is a rough outline on how it might proceed:

v0.1: Introduce a Change Set object

1. Can create a Change Set and link MRs.
1. Change Set will aggregate merge request status from multiple merge requests.
1. Links to MR diffs.
1. API support for retrieving MRs

v0.2: Change Set Merge V1

1. Best effort merging, no guarantees of atomicity.
2. Aggregate reverts across projects.
3. Simplistic permissions: Merge only available to user who can initiate merge for all projects?

v0.3: Code Review with Change Sets V1

1. Change Set diffs aggregated in single view.
1. Possible to see diffs from one specific project.
1. Support of rapid diffs.

v0.4: Change Set Merge V2

1. GitLab will block merges from other merge requests while Change Set merge is happening.
1. Merge commits will be tracked for each merge.
1. Change Set revert: A revert will now be a set of reverts that can be merged together.

v0.5: Change Set pipelines V1

1. Associate a Change Set with a target build project.
1. By default, any push to the Change Set will create a pipeline in the target build project.
1. Affected Change Set projects/files published in a CI variable.

v0.6: CI runner integration with Change Sets

1. Support CI steps or native integration for checking out multiple repositories (e.g. via Android manifest file?).

Known unknowns

This is a list of design aspects that have not yet been refined:

- Is the "product" purely a collection of repositories or does it require additional metadata?
What permissions are required to create a product?
- Define limits. How many MRs can be associated to a single Change Set? How many products can a group or organization host?
- What is the lifecycle of a Change Set?
- Should the system have a way to handle dependency relationships or is it left to the user?
- How will merge conflicts across multiple repositories be detected, reported and resolved?
- Can true atomicity be achieved for merges across repositories, or what level of consistency guarantee can be provided?
- Can the Change Set contain MRs with different merge strategies? How can inconsistencies be checked and reported?
- How can a ChangeSet be used with auto-merge strategies (e.g. Merge When Checks Pass)?
Do we need a mechanism that intercepts and manages the state of each MR to then issue the merge together?
- What permissions are required to perform the merge across multiple projects?
Should customers use service accounts? Note that post-merge CI pipelines will run with that identity.
- How can GitLab provide the merged result pipelines experience in a "poly repo" environment?
- What changes to Glab CLI would be necessary to provide a smooth UX?
- What happens if a partial revert is needed, or if a revert fails in one repository but succeeds in others?
- Should issues be linked to change sets like they are with merge requests?

SECTION: engineering/architecture/design-documents/prompts_migration/_index.md

<!-- Design Documents often contain forward-looking statements -->

<!-- This renders the design document header